

COLLAPSIBLE PANELS

functional specification

IdleWorlds Fantasy Skin → IdleWorlds core. Frozen 2026-09.

WHAT THIS IS

A complete, self-contained description of a shipped feature: every panel on the page folds to a single uniform bar and remembers that it is folded. It was built inside a browser extension that may not touch React, shipped, and tuned against live captures. This file is the distilled result.

HOW TO READ IT

It is pseudo-code, not source. Nothing here imports anything, and none of it is meant to run. Sections marked **DROP** describe work that exists ONLY because the extension could not modify the app. You own the app. Delete them and the feature gets simpler. Sections marked **KEEP** are the ones that cost real time to get right; the numbers in them were measured on the live page, not chosen.

READING ORDER IF YOU ARE SHORT ON BUDGET

1. §1 BEHAVIOUR — what it does. ~40 lines.
 2. §5 THE COLLAPSED BAR — the only genuinely hard part.
 3. §9 MEASURED VALUES — the table. Copy the numbers.
- Everything else is supporting detail.

§1 BEHAVIOUR

Every "panel" — the game's card primitive, the thing with a heading and a body — gains a small chevron button at the right end of its heading row.

Clicking it folds the panel to a ONE-LINE BAR showing only its title, and clicking again restores it exactly. The choice persists per panel, across reloads, forever, until changed.

Three invariants, each of which was violated by an earlier version and each of which produced a bug report:

- I1. A collapsed panel still says WHICH PANEL IT IS. Folding to nothing erases the only thing that made the bar clickable. The title always survives.
- I2. Every collapsed bar is EXACTLY THE SAME HEIGHT as every other. This is the whole feature's perceived quality. See §5 — it is much harder than it sounds and it is where all the effort went.
- I3. Folding destroys no information permanently. Everything hidden returns on expand. Nothing is unmounted in a way that loses scroll position, focus, or in-flight state that the user would notice.

§2 DATA MODEL

```
const StoredState = {
  // One storage key for the whole feature, holding a flat map of
  // panelId -> true. Only COLLAPSED panels are stored; absence means
  // expanded. That choice matters: a new panel the app ships later defaults
  // to expanded, which is the safe direction. Storing `false` explicitly
  // would mean a schema migration every time a panel is added.
  key: 'collapsed-panels',
  shape: { '<panelId>': true },
};
```

KEEP — the default direction. Unknown panel $\Rightarrow$ expanded. Never the reverse. A player who upgrades and finds a panel they have never seen already folded away will report it as missing, not as collapsed.

DROP — panel identity. The extension had to DERIVE an id for each panel from whatever the DOM offered, because the app ships almost no stable attributes. It used, in order: the element id, then the heading text lowercased with any leading emoji stripped, then an internal slug.

You have component identity for free. Give every panel a literal, permanent string id at its definition site and store against that. Do not derive it from the title: the extension had to, and the cost was that RENAMING A PANEL SILENTLY FORGOT ITS STATE.

§3 THE STATE MACHINE

```
function panelCollapseModel(panelId, stored) {
  return {
    collapsed: stored[panelId] === true,

    toggle() {
      this.collapsed = !this.collapsed;
      // Persist IMMEDIATELY on the gesture, not on unload. The game is an idle
      // game left open for hours in a background tab; an unload handler is the
      // least reliable moment to write anything.
      persist(panelId, this.collapsed);
    },
  };
}
```

KEEP — the in-flight read. Storage reads are async. If the player toggles a panel BEFORE the stored map has arrived, the arriving map must not overwrite what they just did. Track the set of ids touched this session and skip those when the read lands.

This is a real race, not a theoretical one: the panels render, the player clicks one within a few hundred milliseconds, and on a slow profile the read has not resolved. The symptom is a panel that folds and then silently re-opens about a second later.

§4 RENDERING — THE SHAPE OF A COLLAPSED PANEL

```
function Panel({ id, title, headerControls, children }) {
  const { collapsed, toggle } = panelCollapseModel(id, stored);

  return Frame({ 'data-collapsed': collapsed },
    HeaderRow(
      // The title is the ONLY thing that survives a fold. Everything else in
      // the header row is hidden with the body – see §5 for why.
      Title({ collapsedTreatment: collapsed }, title),
      collapsed ? null : headerControls,
      CollapseToggle({ expanded: !collapsed, onClick: toggle, label: title }),
    ),
    collapsed ? null : Body(children),
  );
}
```

KEEP — what the header row loses on collapse. Not just the body. The header rows on this page carry, variously: a daily XP boost line and a reset countdown, an installed/total slot count, an XP-per-hour readout and a win rate, a "showing the latest 100 messages" note, and the app's own controls (three inventory tools, four chat icons, a pager, a cancel button). ALL of it folds away. Keeping any of it makes the bars different heights, which breaks I2.

DROP — the "spine". The extension could not conditionally render, so it marked the DOM path from the header row down to the title and hid every element in the row that was not on that path. That machinery exists purely to simulate what you get from `collapsed ? null : headerControls`. Delete it.

§5 THE COLLAPSED BAR — the hard part

KEEP — all of it. This section is the feature.

Reported from a live capture as "very inconsistent". Left alone, collapsed panels are NOT the same height, for four independent reasons. Each has to be neutralised separately, and missing any one of them is visible as a ragged column of bars.

R1. THE TITLES ARRIVE IN DIFFERENT TREATMENTS. Three, on this page: a display face at a fluid size, a second panel whose title is a `<button>` and therefore inherits the UI face, and a third that uppercases. A collapsed bar must impose ONE treatment on all of them.

R2. LINE HEIGHT MUST BE A FIXED PIXEL VALUE, NOT A RATIO. This is the single load-bearing declaration. A ratio multiplies whatever font size wins, so any title that resolves differently produces a different bar height. 20px fixed is what makes them identical.

R3. THE FRAME'S VERTICAL PADDING DIFFERS BY PANEL FAMILY. Measured: 18px on one family, 26px on another. That 8px difference is 16px of bar height. Collapse must level it, not inherit it.

R4. THE HORIZONTAL PADDING DIFFERS TOO. 20px vs 22px. Invisible in isolation; down a column of folded bars it puts the toggle at two different distances from the edge, which reads as misalignment rather than as padding.

The uniform bar is therefore:

```
height      = title line-height (20px fixed)
              + collapsed vertical padding (7px, levelled across families)
              × 2
title       = one font, one size, one weight, one letter-spacing,
              nowrap + ellipsis
padding-x   = levelled across families
```

HOW TO TEST IT — this is the only check shape that can see "inconsistent": fold every panel at once, collect the set of measured bar heights, and require the set to have EXACTLY ONE MEMBER. Do the same for the computed title treatment. Spot-checking one panel cannot detect this class of bug.

§6 THE TOGGLE — placement

KEEP — the geometry and the reason for it.

The control sits at the RIGHT END of the header row, vertically centred ON THE ROW.

Centred on the row, not on the title. One panel carries a second line under its heading and centres its own icon tools on the row; centring on the title puts the toggle visibly high there.

The row must reserve horizontal space for it. Measured, header rows on this page run from 30px to 93px tall, and five of nine panels already have the app's own controls at exactly that end of exactly that row. Without a reserved gutter the toggle lands on top of one of them and the player LOSES a click target rather than gaining one.

```
gutter reserved on the header row : 30px
control box                        : 22 × 22
clearance to the nearest app control: 8px
```

DROP — the absolute positioning. The extension positioned the toggle absolutely so it would not become a third flex item in the app's own `justify-between` header and push the app's controls to the middle. You control that header. Render the toggle as a real child in the correct place and the constraint disappears — but keep the 30px of reserved space, because the collision it prevents is real.

KEEP — the chevron is drawn, not typed. It is a rotated pair of CSS borders, not a glyph. No font dependency, nothing for a text search to match, and it rotates smoothly between states. Expanded points UP and collapsed points DOWN: the chevron shows WHAT THE CLICK WILL DO, not what the panel is currently doing. That is the accordion convention and reversing it reads as a bug.

§7 ORNAMENT ON A COLLAPSED BAR

KEEP if you keep the decorative frame; otherwise DROP the whole section.

The expanded panel draws a filigree in all four corners. A 34px-tall bar cannot: four 34px corners overlap into an unreadable band.

The fix is ONE flourish, top-left only, at the art's NATIVE aspect.

Two mistakes were made here and both are worth knowing:

1. Shrinking the surviving corner to fit. The source quadrant is 88×95, so windowing it to 32×15 stretches it past twice its width-for-height and it renders as a smear. Reported live as "the corner filigree is squashed somehow". With three corners gone there is nothing left to overlap, so there is no reason to shrink it at all.
2. The title then sits ON the flourish. It needs its own left indent — applied to the HEADER ROW, not by reducing the frame's padding, so the bar's left edge still lines up with every other panel in the column.

```
flourish reach from the frame's left edge : ~32px  
title indent applied to the header row    : 22px
```

§8 ACCESSIBILITY

KEEP — all of it, it is cheap.

- The toggle is a real `<button type="button">`, focusable in DOM order.
- `aria-expanded` reflects the state: "true" expanded, "false" collapsed.
- The accessible name names the PANEL and the ACTION, and changes with the state: "Collapse Inventory" / "Expand Inventory". Not "Toggle".
- The title element remains in the accessibility tree when collapsed. It is the bar's only content and the only thing that identifies it.
- The chevron's rotation respects `prefers-reduced-motion`.

NOT NEEDED: `aria-controls`. It requires an id on the body element and buys nothing here, since the button is adjacent to what it controls.

§9 MEASURED VALUES

```
const MEASURED = {  
  // Every number here came off the live page. They are starting points for  
  // your own design system, not magic constants – but the RELATIONSHIPS  
  // between them are what produce a uniform bar.  
  collapsedTitleLineHeight: '20px', // fixed px, NOT a ratio. See §5 R2.  
  collapsedTitleFontSize:   '14px',  
  collapsedPaddingY:        '7px',  // levelled across panel families  
  collapsedPaddingX:        '22px',  // levelled; 16px on narrow viewports  
  resultingBarHeight:       '34px',  
  toggleBox:                '22×22',  
  reservedGutter:           '30px',  // on the header row, for the toggle  
  clearanceToAppControls:   '8px',  
  headerRowRange:           '30px–93px', // why the toggle centres on the ROW  
  framePaddingYByFamily:    '18px / 26px', // the 16px of bar height to level  
  framePaddingXByFamily:    '20px / 22px',  
  chevronTransition:        '160ms ease',  
};
```

§10 FAILURE MODES SEEN IN PRODUCTION

Each of these shipped, was reported, and was fixed. They are listed because a reimplementaion can reproduce every one of them.

F1. "Very inconsistent." Bars at four different heights. Cause: §5 R1–R4, all four at once. Detection: assert the SET of bar heights has one member.

F2. The toggle covered an app control on three panels. Cause: no reserved gutter. Detection: for every panel, assert the toggle's box does not intersect any app-owned control in the same row.

F3. The same control rendered at two different sizes down one column. Cause: a more specific rule elsewhere in the cascade claimed it inside one panel family only. Detection: assert the set of computed control boxes across all panels has one member. This is the same shape of check as F1 and it is the only one that finds this class of bug.

F4. Two of the control's own style declarations never painted at all. Cause: a generic `button` rule elsewhere outranked them, silently, for the life of the feature. Nothing reported it because the existing check only compared the panels to EACH OTHER, and the generic rule won uniformly on all of them. Detection: pin the control's computed appearance to the values it is SUPPOSED to have, not to its neighbours.

F5. A collapsed panel lost its title. Cause: hiding the row's children hid the wrapper the title was inside. In your codebase this is just `collapsed ? null : extras` — but assert the title is present and visible in the collapsed state anyway, because it is invariant I1 and it is one line to check.

§11 WHAT THIS COST, AND WHAT IT SHOULD COST YOU

In the extension: ~270 lines of JS, ~190 lines of CSS, and eight separate opt-outs added to unrelated selectors and classifiers elsewhere, because adding a control inside a node the app owns changes what every other piece of code reading that node sees.

In your codebase this should be: a boolean per panel, one persisted map, one button component, and the §5 bar rules. Call it a day's work including the uniformity tests.

The value in this document is §5, §6, §9 and §10 — the parts that are about what a collapsed panel should LOOK like and how it fails. The mechanism is the easy half and you already have a better one.